

Project Report

Raspberry Pi 5G Network Module Setup

Waveshare RM520N-GL 5G HAT Configuration

1. Project Overview

This project involves setting up and configuring a Waveshare RM520N-GL 5G HAT on a Raspberry Pi 5 to connect to a private 5G laboratory network. The goal is to validate the network connection, assist with debugging, and enable 5G network testing in the lab environment.

2. Components

2.1 Components Inside the Box

Component	Description	Quantity
RM520N-GL 5G Module 	Quectel M.2 B-key 5G module, Qualcomm X62, sub-6GHz global bands	1
M.2 to 4G/5G HAT Board 	Waveshare carrier board for Raspberry Pi, USB 3.0 interface	1
Quad Antennas 	LTE-A MIMO antennas with SMA connectors	4
IPEX Antenna Cables 	Small pigtail cables connecting module to SMA connectors	4
USB 3.0 Dual-Plug port 	Data connection between HAT and Raspberry Pi	1
USB-C Power Cables 	5V 3A external power supply for the 5G module	1

2.2 Additional Items Required

- Raspberry Pi 5
- MicroSD Card (minimum 16GB) for Raspberry Pi OS
- Nano SIM Card — provisioned for the lab's private 5G network
- 5V 3A USB-C Power Adapter — for external HAT power
- Laptop/PC — for OS flashing and SSH access



3. Product Reference

Product: RM520N-GL 5G HAT for Raspberry Pi with Case, Quad Antennas LTE-A, Global Band, GNSS Positioning, Support 3GPP Release 16, 4G/3G

Manufacturer: Waveshare Electronics

Product Link: <https://www.waveshare.com/rm520n-gl-5g-hat-with-case.htm>

Wiki/Documentation: https://www.waveshare.com/wiki/RM520N-GL_5G_HAT

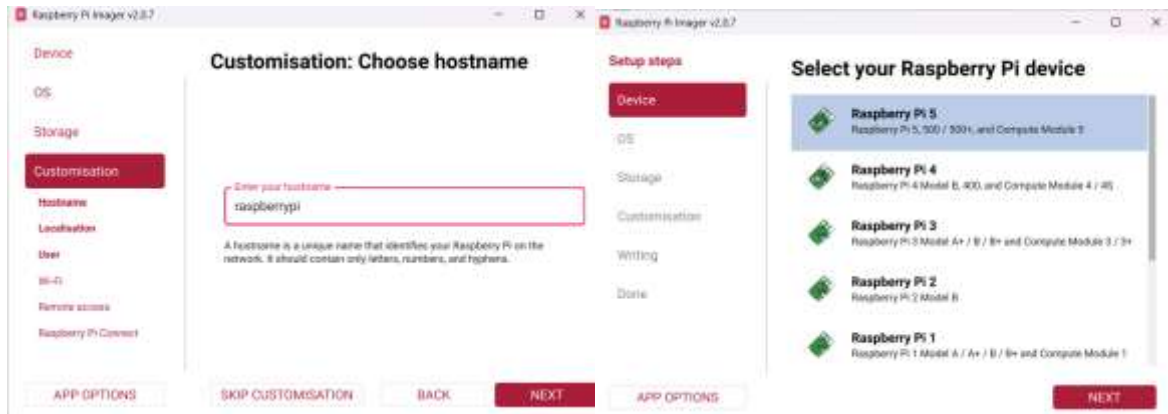
4. Step-by-Step Setup Process

• Prepare MicroSD Card

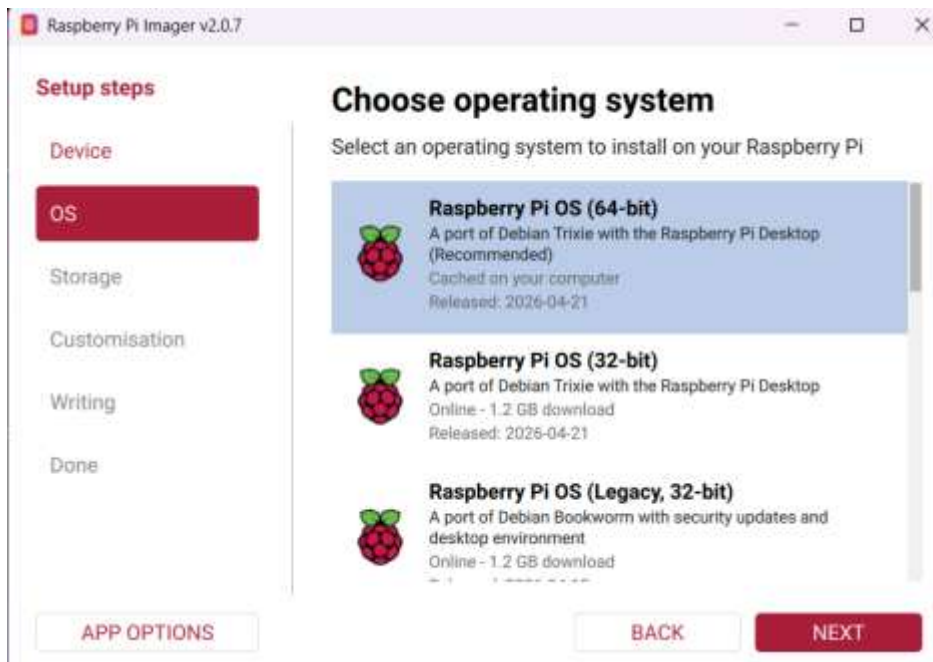
1. Download and install Raspberry Pi Imager v2.0.7 on laptop from <https://www.raspberrypi.com/software/>



2. Open Raspberry Pi Imager and select device: Raspberry Pi 5



3. Select OS: Raspberry Pi OS (64-bit)



4. Insert MicroSD card into USB card reader and connect to laptop



5. Select the MicroSD card as storage device

The screenshot shows the 'Customisation: Localisation' screen in the Raspberry Pi Imager v2.0.7. The left sidebar has 'Customisation' selected. The main area has the title 'Customisation: Localisation' and the instruction 'Select your location for suggested time zone and keyboard layout'. There are three dropdown menus: 'Capital city:' set to 'Helsinki (Finland)', 'Time zone:' set to 'Europe/Helsinki', and 'Keyboard layout:' set to 'fi'. At the bottom are four buttons: 'APP OPTIONS', 'SKIP CUSTOMISATION', 'BACK', and 'NEXT'.

6. In Customization settings: set hostname (raspberrypi), username (pi), password, Wi-Fi credentials

The first screenshot shows the 'Customisation: Choose username' screen. The left sidebar has 'Customisation' selected. The main area has the title 'Customisation: Choose username' and the instruction 'Create a user account for your Raspberry Pi'. There are three input fields: 'Username:' with 'pi', 'Password:' with 'raspberrypi123', and 'Confirm password:' with '*****'. A note at the bottom states: 'The username must be lowercase and contain only letters, numbers, underscores, and hyphens.' The second screenshot shows the 'Customisation: Choose Wi-Fi' screen. The left sidebar has 'Customisation' selected. The main area has the title 'Customisation: Choose Wi-Fi' and two tabs: 'SECURE NETWORK' and 'OPEN NETWORK'. There are three input fields: 'SSID:' with 'AloTatangers', 'Password:' with 'AloT-2024', and 'Confirm password:' with '*****'. A checkbox 'Hidden SSID' is unchecked. Both screens have 'APP OPTIONS', 'SKIP CUSTOMISATION', 'BACK', and 'NEXT' buttons at the bottom.

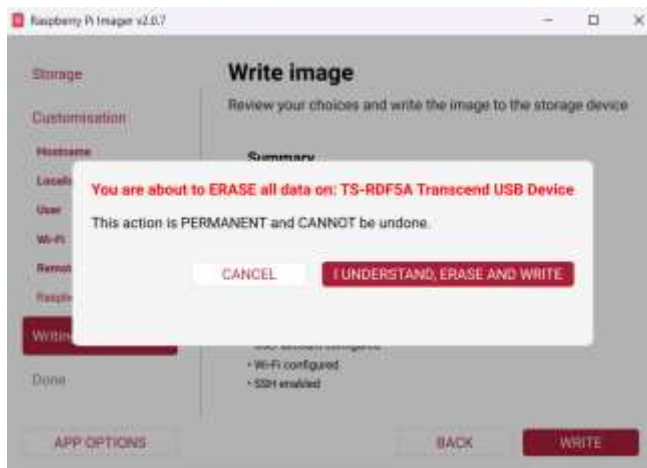
7. Enable SSH under Remote Access — select 'Use password authentication'

The screenshot shows the 'Customisation: SSH authentication' screen in the Raspberry Pi Imager v2.0.7. The left sidebar has 'Customisation' selected. The main area has the title 'Customisation: SSH authentication' and the instruction 'Configure SSH access'. There is a toggle switch 'Enable SSH' which is turned on. Below it, there are two radio buttons for 'Authentication mechanism': 'Use password authentication' (selected) and 'Use public key authentication'. At the bottom are four buttons: 'APP OPTIONS', 'SKIP CUSTOMISATION', 'BACK', and 'NEXT'.

8. Leave Raspberry Pi Connect disabled

The screenshot shows the 'Customisation: Raspberry Pi Connect' screen in the Raspberry Pi Imager v2.0.7. The left sidebar has 'Customisation' selected. The main area has the title 'Customisation: Raspberry Pi Connect' and the instruction 'Sign in to receive a token and enable Raspberry Pi Connect'. There is a toggle switch 'Enable Raspberry Pi Connect' which is turned off. At the bottom are four buttons: 'APP OPTIONS', 'SKIP CUSTOMISATION', 'BACK', and 'NEXT'.

9. Click Write and confirm — wait 5-10 minutes for completion



10. Eject MicroSD card safely and insert into Raspberry Pi 5



- **Hardware Assembly**

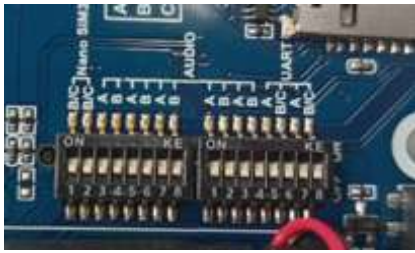
11. Remove RM520N-GL module from anti-static bag - handle by edges only



12. Insert module into M.2 slot on HAT at 30-degree angle, press flat, secure with screw



13. Set DIP switches: C = ON, A = OFF, B = OFF (for RM520N-GL / RM50X series)



14. Connected IPEX antenna cables from module ports to HAT SMA connectors

15. Attach all 4 antennas to SMA connectors

16. Insert Nano SIM card into SIM1 slot

17. Connect HAT to Raspberry Pi 5 via USB 3.0 dual-plug cable (blue USB port on Pi)

18. Set HAT power switch to EXT PWR position

19. Connect 5V 3A USB-C power adapter to HAT EXT PWR port



• SSH Connection

20. Power on the Raspberry Pi 5 and Wait 60 seconds for full boots

21. Verify HAT LEDs: PWR LED = red (module powered), NET LED = green (network acquired)

22. Open terminal cmd or powershell and Located Pi on network using *ping raspberry.pi.local*

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26290.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shami>ping raspberry.pi.local

Pinging raspberry.pi.local [fe80::8aa2:9eff:fe5f:89e3%9] with 32 bytes of data:
Reply from fe80::8aa2:9eff:fe5f:89e3%9: time=6ms
Reply from fe80::8aa2:9eff:fe5f:89e3%9: time=18ms
Reply from fe80::8aa2:9eff:fe5f:89e3%9: time=16ms
Reply from fe80::8aa2:9eff:fe5f:89e3%9: time=17ms

Ping statistics for fe80::8aa2:9eff:fe5f:89e3%9:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 6ms, Maximum = 18ms, Average = 14ms

C:\Users\shami>
```

23. Clear old SSH host keys using `ssh-keygen -R raspberrypi.local`

```
C:\Users\shami>ssh-keygen -R raspberrypi.local
# Host raspberrypi.local found: line 28
C:\Users\shami/.ssh/known_hosts updated.
Original contents retained as C:\Users\shami/.ssh/known_hosts.old
```

24. Successfully SSH into Pi: `ssh pi@raspberrypi.local`

```
C:\Users\shami>ssh pi@raspberrypi.local
The authenticity of host 'raspberrypi.local ([fe80::8aa2:9eff:fe5f:89e349])' can't be established.
ED25519 key fingerprint is SHA256:j5saTrvMyan@wNG6E0Z7LYD8E+4RotyBwvchJF3Zz6Q.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'raspberrypi.local' (ED25519) to the list of known hosts.
pi@raspberrypi.local's password:
Linux raspberrypi 6.12.75+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.12.75-1+rpt1 (2026-03-11) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
pi@raspberrypi:~$
```

25. Enter password when prompted

- **System Update and Preparation**

26. Update OS: `sudo apt update && sudo apt full-upgrade -y`

```
pi@raspberrypi:~$ sudo apt update && sudo apt full-upgrade -y
[sudo] password for pi:
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://archive.raspberrypi.com/debian trixie InRelease [54.8 kB]
Get:5 http://deb.debian.org/debian trixie/main armhf Packages [9,238 kB]
Get:6 http://archive.raspberrypi.com/debian trixie/main armhf Packages [451 kB]
Get:7 http://archive.raspberrypi.com/debian trixie/main arm64 Packages [458 kB]
Get:8 http://deb.debian.org/debian trixie/main arm64 Packages [9,608 kB]
Get:9 http://deb.debian.org/debian trixie/main Translation-en [6,485 kB]
Get:10 http://deb.debian.org/debian-security trixie-security/main arm64 Packages [208 kB]
Get:11 http://deb.debian.org/debian-security trixie-security/main armhf Packages [189 kB]
Get:12 http://deb.debian.org/debian-security trixie-security/main Translation-en [126 kB]
Fetched 26.8 MB in 4s (6,518 kB/s)
Processing triggers for mailcap (3.74) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for hicolor-icon-theme (0.18-2) ...
Processing triggers for libc-bin (2.41-12+rpt1+deb13u2) ...
Processing triggers for systemd (257.13-1+deb13u1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for initramfs-tools (0.148.4+rpt2) ...
update-initramfs: Generating /boot/initrd.img-6.18.33+rpt-rpi-v8
'/boot/initrd.img-6.18.33+rpt-rpi-v8' -> '/boot/firmware/initramfs8'
update-initramfs: Generating /boot/initrd.img-6.18.33+rpt-rpi-2712
'/boot/initrd.img-6.18.33+rpt-rpi-2712' -> '/boot/firmware/initramfs_2712'
pi@raspberrypi:~$ sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

27. Installed dhcpcd5 for WiFi management: *sudo apt install dhcpcd5 -y*

```
pi@raspberrypi:~ $ sudo apt install dhcpcd5 -y
[sudo] password for pi:
Installing:
  dhcpcd5

Installing dependencies:
  dhcpcd

Suggested packages:
  dhcpcd-gtk

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 44
  Download size: 25.7 kB
  Space needed: 63.5 kB / 110 GB available
```

28. Configured wpa_supplicant.conf with lab WiFi credentials

```
pi@raspberrypi: ~
GNU nano 8.4 /etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=FI

network={
    ssid="AIoT-strangers"
    psk="AIoT-2024"
```

29. Remove conflicting software: *sudo apt purge modemmanager -y*

```
pi@raspberrypi:~ $ sudo apt purge modemmanager -y
Package 'modemmanager' is not installed, so not removed
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- **5G Module Detection and Configuration**

30. Check USB device detection: *usb-devices*

31. Verify serial ports created: *ls /dev/ttyUSB**

```
pi@raspberrypi:~ $ ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1 /dev/ttyUSB2 /dev/ttyUSB3
```

32. Install minicom: *sudo apt-get install minicom -y*

```
pi@raspberrypi:~$ sudo apt-get install minicom -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  lrzsz
The following NEW packages will be installed:
  lrzsz minicom
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 330 kB of archives.
After this operation, 1,506 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main arm64 lrzsz arm64 0.12.21rc-0.2 [79.7 kB]
Get:2 http://deb.debian.org/debian trixie/main arm64 minicom arm64 2.10-1 [250 kB]
Fetched 330 kB in 0s (1,080 kB/s)
Selecting previously unselected package lrzsz.
(Reading database ... 141883 files and directories currently installed.)
Preparing to unpack .../lrzsz_0.12.21rc-0.2_arm64.deb ...
Unpacking lrzsz (0.12.21rc-0.2) ...
Selecting previously unselected package minicom.
Preparing to unpack .../minicom_2.10-1_arm64.deb ...
Unpacking minicom (2.10-1) ...
Setting up minicom (2.10-1) ...
Setting up lrzsz (0.12.21rc-0.2) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
```

33. Open AT command port: *sudo minicom -D /dev/ttyUSB2*

```
pi@raspberrypi:~$ sudo minicom -D /dev/ttyUSB2

Welcome to minicom 2.10

OPTIONS: I18n
Port /dev/ttyUSB2 [U]

Press CTRL-A Z for help on special keys

AT
OK
AT+CPIN?
+CPIN: READY

OK
AT+CEREG?
+CEREG: 0,0

OK
AT+C5GREG?
+C5GREG: 0,1

OK
AT+COPS?
+COPS: 0,0,"FXR",11

OK
AT+QCFG="usbnet",1
OK
AT+CFUN=1,1
OK

RDY

+CPIN: READY

+QUSIM: 1

+CFUN: 1

+QIND: SMS DONE

+QIND: PB DONE
```

34. Tested basic communication: AT command returned OK
35. Verified SIM card: AT+CPIN? returned +CPIN: READY
36. Checked LTE registration: AT+CEREG? returned 0,0 (not registered on LTE)
37. Checked 5G registration: AT+C5GREG? returned 0,1 (registered on 5G)
38. Verified operator: AT+COPS? returned FXR on network type 11 (5G NR)
39. Set USB network mode to ECM: AT+QCFG="usbnet",1
40. Restarted module: AT+CFUN=1,1 - module returned RDY
41. Exit minicom: Ctrl+A then X

- **Connect to 5G Network**

42. Verified new network interface: *ip addr show*, usb0 appeared with IP 192.168.225.97
43. Wi-Fi interface wlan0 confirmed UP with IP 192.168.50.48

```
pi@raspberrypi:~ $ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 88:a2:9e:5f:89:e2 brd ff:ff:ff:ff:ff:ff
4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 88:a2:9e:5f:89:e3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.48/24 brd 192.168.50.255 scope global dynamic noprefixroute wlan0
        valid_lft 84665sec preferred_lft 84665sec
    inet6 fe80::8aa2:9eff:fe5f:89e3/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: usb0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether e2:ba:14:7b:fc:1f brd ff:ff:ff:ff:ff:ff
    inet 192.168.225.97/22 brd 192.168.227.255 scope global dynamic noprefixroute usb0
        valid_lft 42613sec preferred_lft 42613sec
    inet6 fe80::e2ba:14ff:fe7b:fc1f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

44. Tested 5G connectivity: *ping -I usb0 8.8.8.8 -c 4* — 0% packet loss

```
pi@raspberrypi:~ $ ping -I usb0 8.8.8.8 -c 4
PING 8.8.8.8 (8.8.8.8) from 192.168.225.97 usb0: 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=179 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=9.14 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=10.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=9.13 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 9.129/51.775/178.569/73.205 ms
pi@raspberrypi:~ $ |
```

45. Tested wifi enabled: `ping -I wlan0 8.8.8.8 -c 4` — 0% packet loss

```
pi@raspberrypi:~ $ ping -I wlan0 8.8.8.8 -c 4
PING 8.8.8.8 (8.8.8.8) from 192.168.50.48 wlan0: 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=41.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=15.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=114 time=15.9 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=114 time=56.9 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 15.905/32.461/56.925/17.470 ms
```

46. Test 5G works independently with WiFi disabled: `sudo ip link set wlan0 down && ping -I usb0 8.8.8.8 -c 4 && sudo ip link set wlan0 up`

```
pi@raspberrypi:~ $ sudo ip link set wlan0 down && ping -I usb0 8.8.8.8 -c 4 && sudo ip link set wlan0 up
PING 8.8.8.8 (8.8.8.8) from 192.168.225.97 usb0: 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=12.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=9.25 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=8.75 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=9.16 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 8.745/9.871/12.330/1.432 ms
```

47. Check 5G signal: `AT+QENG="servingcell" - NR5G-SA, band n77, RSRP -72, SINR 19-30`

```
pi@raspberrypi:~ $ sudo minicon -D /dev/ttyUSB2
[sudo] password for pi:

Welcome to minicon 2.10

OPTIONS: I18n
Port /dev/ttyUSB2, 10:40:35 [U]

Press CTRL-A Z for help on special keys

AT+QENG="servingcell"
+QENG: "servingcell", "NOCONN", "NR5G-SA", "TDD", 244, 38, 000FB4001, 150, 3E8, 663264, 77, 12, -72, -10, 26, 1, -
OK
```

48. Run speed test through 5G: Download 182.98 Mbps, Upload 154.99 Mbps, Ping 10ms

`curl -s https://raw.githubusercontent.com/sivel/speedtest-cli/master/speedtest.py | python3 --simple --source 192.168.225.97`

```
pi@raspberrypi:~ $ curl -s https://raw.githubusercontent.com/sivel/speedtest-cli/master/speedtest.py | python3 --simple --source 192.168.225.97
<stdin>:900: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to repr
sent datetimes in UTC: datetime.datetime.now(datetime.UTC).
Ping: 10.071 ms
Download: 182.98 Mbit/s
Upload: 154.99 Mbit/s
```

The Raspberry Pi 5 is successfully connected to the private 5G network with IP 192.168.225.97 and internet connectivity confirmed through the 5G interface

5. 5G Connection Validation Results

- **5G Signal Information** (Command used: AT+QENG="servingcell")

Parameter	value	Meaning
Network Type	NR5G-SA	True 5G Standalone network
Mode	TDD	Time Division Duplex
Country Code	244	Finland
Operator	FXR	Lab private 5G network
Band	N77	5G NR band 77
Cell Tower ID	000FB4001	Specific 5G base station
RSRP	-67 to -73 dbm	Good signal strength
SINR	19 – 33 dB	Excellent signal quality
Network Type Code	11	5G NR confirmed

- **Speed Test Results**

Metric	5G (usb0)	WiFi (wlan0)	Winner
Ping	10-12 ms	15 -56 ms	5G is 3x faster
Download Speed	182.98 Mbps	Not tested	5G
Upload Speed	154.99 Mbps	Not tested	5G
Packet Loss	0 %	0%	Equal

- 5G module detected on ttyUSB0, ttyUSB1, ttyUSB2, ttyUSB3
- SIM card recognized - AT+CPIN? returned READY
- Registered on 5G NR Standalone network (operator FXR, band n77, Finland)
- sb0 interface created with IP 192.168.225.97

- Internet connectivity confirmed through 5G — 0% packet loss
- Download speed: 182.98 Mbps via 5G
- Upload speed: 154.99 Mbps via 5G
- 5G works independently when WiFi is completely disabled
- WiFi (wlan0) and 5G (usb0) operate simultaneously without conflict

6. Network Traffic Logging and Analysis

tshark is the command-line version of Wireshark. It captures all network packets on a specified interface and saves them to a .pcap file that can be analyzed in detail.

• Installation and Setup

1. Install tshark on Raspberry Pi: *sudo apt-get install tshark -y*

```
C:\Users\shami>ssh pi@raspberrypi.local
pi@raspberrypi.local's password:
Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 10 11:44:12 2026 from fe80::88ae:58e4:418:a6de%wlan0
pi@raspberrypi:~$ sudo apt-get install tshark -y
[sudo] password for pi:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libbcg729-0 libcares2 libmaxminddb0 libsn12t64 libwireshark-data libwireshark18 libwiretap15 libwsutil16 wireshark-common
Suggested packages:
  mdb-bin snmp-mibs-downloader geoipupdate geoip-database geoip-database-extra libjs-leaflet libjs-leaflet.markercluster wireshark-doc
The following NEW packages will be installed:
  libbcg729-0 libcares2 libmaxminddb0 libsn12t64 libwireshark-data libwireshark18 libwiretap15 libwsutil16 tshark wireshark-common
0 upgraded, 10 newly installed, 0 to remove and 0 not upgraded.
```

```
pi@raspberrypi:~$ tshark --version
TShark (Wireshark) 4.4.15.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compiled (64-bit) using GCC 14.2.0, with GLib 2.84.4, with libpcap, with POSIX
capabilities (Linux), with libnl 3, with zlib 1.3.1, without zlib-ng, with
PCRE2, with Lua 5.4.7, with GnuTLS 3.8.9 and PKCS #11 support, with Gcrypt
1.11.0, with Merberos (MIT), with MaxMind, with nghttp2 1.64.0, with nghttp3
1.8.0, with brotli, with LZ4, with Zstandard, with Snappy, with libxml2 2.9.14,
with libsmi 0.4.8, with binary plugins.

Running on Linux 6.18.33+rpt-rpi-2712, with 16218 MB of physical memory, with
GLib 2.84.4, with libpcap 1.10.5 (with IPACKET_V3), with zlib 1.3.1, with PCRE2
10.46 2025-08-27, with c-ares 1.34.5, with GnuTLS 3.8.9, with Gcrypt 1.11.0,
with nghttp2 1.64.0, with nghttp3 1.8.0, with brotli 1.1.0, with LZ4 1.10.0,
with Zstandard 1.5.7, with libsmi 0.4.8, with LC_TYPE=en_GB.UTF-8, binary
plugins supported.
```

2. Add pi user to wireshark group for permission:

```
sudo usermod -a -G wireshark pi
```

```
exit
```

```
ssh pi@raspberrypi.local
```

```

pi@raspberrypi:~$ sudo usermod -a -G wireshark pi
pi@raspberrypi:~$ exit
logout
Connection to raspberrypi.local closed.

C:\Users\shami>ssh pi@raspberrypi.local
pi@raspberrypi.local's password:
Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 10 13:06:43 2026 from fe80::88ae:58e4:418:a6de%wlan0
pi@raspberrypi:~$ |

```

3. Fix dumpcap permissions:

`sudo setcap cap_net_raw,cap_net_admin+eip /usr/bin/dumpcap`

4. Verify tshark can see the 5G interface:

`tshark -D`

```

pi@raspberrypi:~$ sudo setcap cap_net_raw,cap_net_admin+eip /usr/bin/dumpcap
pi@raspberrypi:~$ tshark -D
1. usb0
2. wlan0
3. any
4. lo (Loopback)
5. eth0
6. bluetooth0
7. bluetooth-monitor
8. nflog
9. nfqueue
10. dbus-system
11. dbus-session
12. ciscodump (Cisco remote capture)
13. dpauxmon (DisplayPort AUX channel monitor capture)
14. randpkt (Random packet generator)
15. sdjournal (systemd Journal Export)
16. sshdump (SSH remote capture)
17. udpdump (UDP Listener remote capture)
18. wifidump (Wi-Fi remote capture)
pi@raspberrypi:~$ |

```

usb0 appears as interface number 1 and wlan0 = WiFi interface (number 2)

• The network_logger.sh Script

Create a folder to save logs, network_logs and create the logging script network_logger.sh and make it executable

```

pi@raspberrypi:~$ mkdir -p /home/pi/network_logs
pi@raspberrypi:~$ sudo nano /home/pi/network_logger.sh
[sudo] password for pi:
pi@raspberrypi:~$ chmod +x /home/pi/network_logger.sh
pi@raspberrypi:~$ |

```

This script was created to capture packets, log ping results, and record interface status automatically.

Script location: /home/pi/network_logger.sh

Log location: /home/pi/network_logs/

pcap files: /home/pi/network_logs/5g_traffic.pcap

The script,

- Captures all packets on usb0 (5G interface) using tshark
- Logs ping results to 8.8.8.8 every 5 seconds
- Records interface IP and routing table every 10 seconds
- Saves everything to timestamped log files

• Traffic Analysis Results

After capturing traffic using tshark, the pcap file was analysed using:

tshark -r /home/pi/network_logs/5g_traffic_20260610_132847.pcap

```
pi@raspberrypi:~$ tshark -r /home/pi/network_logs/5g_traffic_20260610_132847.pcap
1 0.000000000 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 Who has 192.168.225.1? Tell 192.168.225.97
2 0.001517984 a6:ef:c7:83:63:8a > 9a:55:e0:8c:a6:a6 ARP 42 192.168.225.1 is at a6:ef:c7:83:63:8a
3 0.002733413 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala3, seq=1/256, ttl=64
4 0.012880528 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala3, seq=1/256, ttl=115 (request in 3)
5 5.016699091 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala4, seq=1/256, ttl=64
6 5.028475542 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala4, seq=1/256, ttl=115 (request in 5)
7 9.192895928 a6:ef:c7:83:63:8a > Broadcast ARP 42 Who has 192.168.225.97? Tell 192.168.225.1
8 9.192913668 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 192.168.225.97 is at 9a:55:e0:8c:a6:a6
9 10.032276513 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala5, seq=1/256, ttl=64
10 10.042567332 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala5, seq=1/256, ttl=115 (request in 9)
11 15.046405821 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala6, seq=1/256, ttl=64
12 15.057284400 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala6, seq=1/256, ttl=115 (request in 11)
13 20.061164815 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala7, seq=1/256, ttl=64
14 20.073132915 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala7, seq=1/256, ttl=115 (request in 13)
15 25.076892219 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala8, seq=1/256, ttl=64
16 25.088718022 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala8, seq=1/256, ttl=115 (request in 15)
17 30.092644512 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xala9, seq=1/256, ttl=64
18 30.102934071 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xala9, seq=1/256, ttl=115 (request in 17)
19 33.513178188 a6:ef:c7:83:63:8a > Broadcast ARP 42 Who has 192.168.225.97? Tell 192.168.225.1
20 33.513194614 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 192.168.225.97 is at 9a:55:e0:8c:a6:a6
21 35.106769116 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalaa, seq=1/256, ttl=64
22 35.121903519 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalaa, seq=1/256, ttl=115 (request in 21)
23 35.327996131 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 Who has 192.168.225.1? Tell 192.168.225.97
24 35.3279538264 a6:ef:c7:83:63:8a > 9a:55:e0:8c:a6:a6 ARP 42 192.168.225.1 is at a6:ef:c7:83:63:8a
25 40.125844731 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalab, seq=1/256, ttl=64
26 40.136367550 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalab, seq=1/256, ttl=115 (request in 25)
27 45.140180095 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalac, seq=1/256, ttl=64
28 45.150834525 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalac, seq=1/256, ttl=115 (request in 27)
29 50.154778033 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalad, seq=1/256, ttl=64
30 50.165426408 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalad, seq=1/256, ttl=115 (request in 29)
31 55.169263730 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalae, seq=1/256, ttl=64
32 55.181268478 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalae, seq=1/256, ttl=115 (request in 31)
33 57.833335826 a6:ef:c7:83:63:8a > Broadcast ARP 42 Who has 192.168.225.97? Tell 192.168.225.1
34 57.833352770 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 192.168.225.97 is at 9a:55:e0:8c:a6:a6
35 60.185232579 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalaf, seq=1/256, ttl=64
36 60.195863670 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalaf, seq=1/256, ttl=115 (request in 35)
37 65.199789017 192.168.225.97 > 8.8.8.8 ICMP 98 Echo (ping) request id=0xalb0, seq=1/256, ttl=64
38 65.212451026 8.8.8.8 > 192.168.225.97 ICMP 98 Echo (ping) reply id=0xalb0, seq=1/256, ttl=115 (request in 37)
39 65.279996043 9a:55:e0:8c:a6:a6 > a6:ef:c7:83:63:8a ARP 42 Who has 192.168.225.1? Tell 192.168.225.97
40 65.281578935 a6:ef:c7:83:63:8a > 9a:55:e0:8c:a6:a6 ARP 42 192.168.225.1 is at a6:ef:c7:83:63:8a
```

...

The complete communication timeline:

- Second 0.000 - Pi finds 5G gateway (ARP handshake: Pi asks, "Who has 192.168.225.1?")
- Second 0.001 - Gateway replies "I am here!" (ARP reply)
- Second 0.002 - Pi pings Google (8.8.8.8) through 5G

- Second 0.012 - Google replies in 10ms
- Every 5 seconds - Pi keeps pinging Google - all 58 pings successful
- Second 9.19 - Gateway checks Pi is still there (ARP broadcast)
- Second 9.19 - Pi confirms it is still there
- Every ~25-35 seconds - Pi and gateway exchange ARP handshakes to keep connection alive
- Second 133.99 - IPv6 Router Solicitation (ICMPv6) - confirms dual-stack IPv4 and IPv6 support
- Second 185.19 - Pi syncs its clock through 5G (NTP request to Finnish server 95.217.203.53)
- Second 185.20 - Time server replies with current time
- Throughout 215 seconds - ZERO failures, ZERO lost packets, 119 packets total

- **Packets Being Sent FROM Raspberry Pi TO 5G Network**

Packet Type	Command Used	Frequency	Purpose
ARP Request	9a:55:e0:8c:a6:a6 → Who has 192.168.225.1? Tell 192.168.225.97	Every 25-35 seconds	Pi asks where the 5G gateway is
ICMP Echo Request	192.168.225.97 → 8.8.8.8	Every 5 seconds	Pi pings internet to test connectivity
NTP Request	192.168.225.97 → 95.217.203.53	Once at second 185	Pi requests time sync via 5G
ICMPv6 Router Solicitation	fe80::6c4c:... → ff02::2	On connect	Pi checks if IPv6 router is available on 5G network

- **Packets Coming BACK FROM 5G Network TO Raspberry Pi**

Packet type	Response	Meaning
ARP Reply	a6:ef:c7:83:63:8a → 9a:55:e0:8c:a6:a6: 192.168.225.1 is at a6:ef:c7:83:63:8a	5G gateway confirms it is reachable
ICMP Echo Reply	8.8.8.8 → 192.168.225.97: Reply	Internet confirms connectivity — avg 10ms
NTP Reply	95.217.203.53 → 192.168.225.97: Time data	Time server responds with current time
ARP Broadcast	a6:ef:c7:83:63:8a → Broadcast: Who has 192.168.225.97? Tell 192.168.225.1	5G gateway checks Pi is still connected

- **Handshakes Between Device and 5G Network**

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
2	0.000117084	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
3	0.192895920	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
4	0.192895968	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
5	0.351178188	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
6	0.351178624	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
7	0.351179120	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
8	0.351179556	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
9	0.351179992	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
10	0.351180428	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
11	0.351180864	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
12	0.351181300	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
13	0.351181736	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
14	0.351182172	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
15	0.351182608	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
16	0.351183044	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
17	0.351183480	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
18	0.351183916	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
19	0.351184352	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
20	0.351184788	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
21	0.351185224	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
22	0.351185660	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
23	0.351186096	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
24	0.351186532	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
25	0.351186968	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
26	0.351187404	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
27	0.351187840	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
28	0.351188276	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
29	0.351188712	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
30	0.351189148	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
31	0.351189584	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
32	0.351189920	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
33	0.351190356	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
34	0.351190792	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
35	0.351191228	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
36	0.351191664	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
37	0.351192100	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
38	0.351192536	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
39	0.351192972	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
40	0.351193408	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
41	0.351193844	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
42	0.351194280	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
43	0.351194716	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
44	0.351195152	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
45	0.351195588	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
46	0.351196024	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
47	0.351196460	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
48	0.351196896	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
49	0.351197332	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
50	0.351197768	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
51	0.351198204	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
52	0.351198640	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
53	0.351199076	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
54	0.351199512	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
55	0.351199948	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
56	0.351200384	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
57	0.351200820	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
58	0.351201256	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
59	0.351201692	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
60	0.351202128	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
61	0.351202564	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
62	0.351203000	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
63	0.351203436	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
64	0.351203872	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
65	0.351204308	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
66	0.351204744	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
67	0.351205180	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
68	0.351205616	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
69	0.351206052	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
70	0.351206488	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
71	0.351206924	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
72	0.351207360	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
73	0.351207796	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
74	0.351208232	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
75	0.351208668	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
76	0.351209104	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
77	0.351209540	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
78	0.351209976	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
79	0.351210412	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
80	0.351210848	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
81	0.351211284	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
82	0.351211720	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
83	0.351212156	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
84	0.351212592	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
85	0.351213028	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
86	0.351213464	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
87	0.351213900	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
88	0.351214336	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
89	0.351214772	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
90	0.351215208	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
91	0.351215644	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
92	0.351216080	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
93	0.351216516	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
94	0.351216952	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
95	0.351217388	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
96	0.351217824	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
97	0.351218260	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
98	0.351218696	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
99	0.351219132	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
100	0.351219568	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
101	0.351220004	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
102	0.351220440	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
103	0.351220876	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	Who has 192.168.225.1? Tell 192.168.225.97
104	0.351221312	a6:ef:c7:83:63:8a	9a:55:e0:8c:a6:a6	ARP	42	192.168.225.1 is at a6:ef:c7:83:63:8a
105	0.351221748	a6:ef:c7:83:63:8a	Broadcast	ARP	42	Who has 192.168.225.97? Tell 192.168.225.1
106	0.351222184	9a:55:e0:8c:a6:a6	a6:ef:c7:83:63:8a	ARP	42	192.168.225.97 is at 9a:55:e0:8c:a6:a6
107	0.351.					

The following handshake pattern was observed every 30 seconds:

Step 1: Pi sends ARP Request: 'Who has 192.168.225.1? Tell 192.168.225.97'

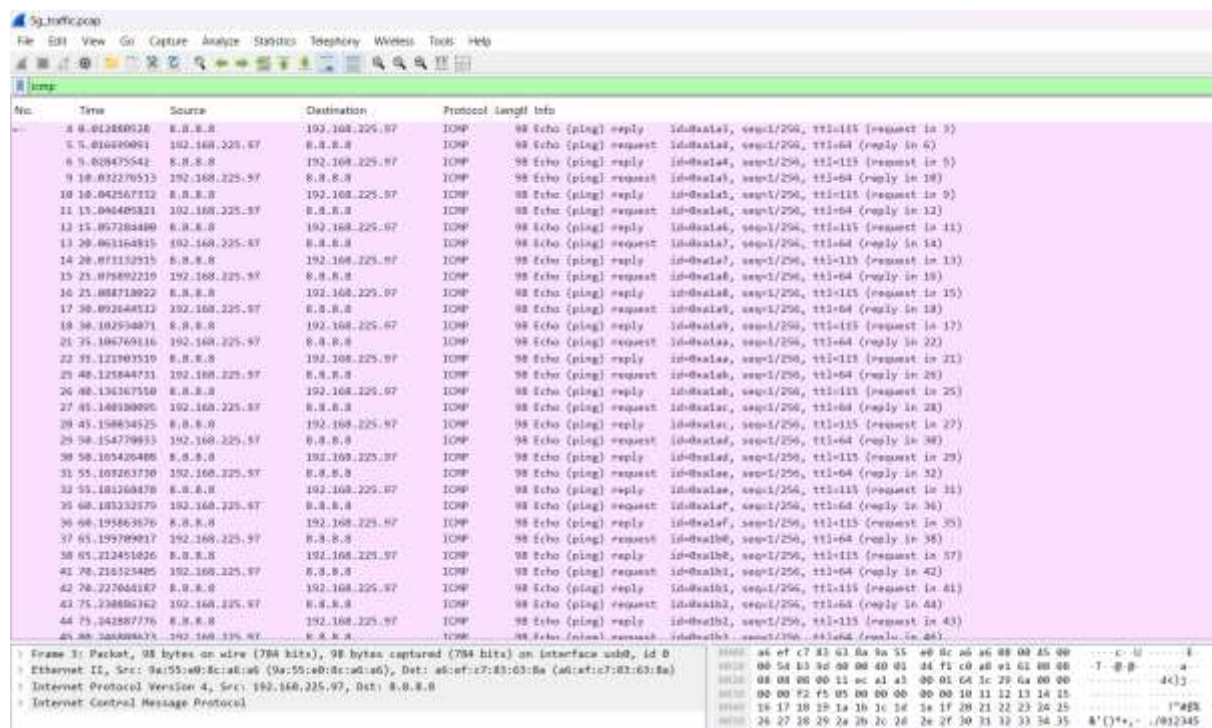
Step 2: Gateway sends ARP Reply: '192.168.225.1 is at a6:ef:c7:83:63:8a'

Step 3: Gateway sends ARP Broadcast: 'Who has 192.168.225.97? Tell 192.168.225.1'

Step 4: Pi responds: '192.168.225.97 is at 9a:55:e0:8c:a6:a6'

This ARP handshake cycle confirms:

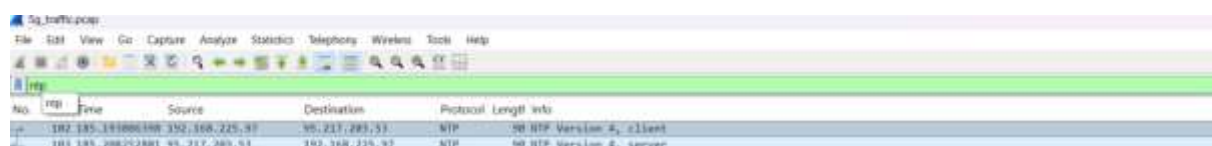
- Both Pi and 5G gateway are actively communicating
- The connection is being kept alive every 30 seconds
- Both devices know each other's MAC addresses
- The 5G network is monitoring the Pi connection actively



The screenshot shows a Wireshark packet capture of ICMP ping traffic. The packet list on the left shows a series of ping requests and replies. The packet details pane on the right shows the structure of a ping request packet, including the Ethernet II header, Internet Protocol Version 4 header, and Internet Control Message Protocol header. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
2	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
3	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
4	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
5	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
6	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
7	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
8	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
9	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
10	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
11	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
12	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
13	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
14	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
15	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
16	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
17	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
18	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
19	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
20	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
21	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
22	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
23	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
24	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
25	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
26	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
27	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
28	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
29	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
30	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
31	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
32	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
33	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
34	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
35	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
36	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
37	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
38	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
39	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
40	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
41	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
42	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
43	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
44	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
45	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
46	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
47	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
48	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)
49	0.000000	192.168.225.97	192.168.225.1	ICMP	60	Echo (ping) request 192.168.225.1 (request in 0)
50	0.000000	192.168.225.1	192.168.225.97	ICMP	60	Echo (ping) reply 192.168.225.1 (reply in 0)

Frame 3: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface usb0, id 0
Ethernet II, Src: 9a:55:e0:8c:a6:a6 (9a:55:e0:8c:a6:a6), Dst: a6:ef:c7:83:63:8a (a6:ef:c7:83:63:8a)
Internet Protocol Version 4, Src: 192.168.225.97, Dst: 8.8.8.8
Internet Control Message Protocol



The screenshot shows a Wireshark packet capture of NTP traffic. The packet list on the left shows two NTP packets: a client request and a server response. The packet details pane on the right shows the structure of an NTP packet, including the Ethernet II header, Internet Protocol Version 4 header, and Network Time Protocol header. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.225.97	192.168.225.1	NTP	60	NTP Version 4, client
2	0.000000	192.168.225.1	192.168.225.97	NTP	60	NTP Version 4, server

The pcap analysis shows the Raspberry Pi maintains continuous ARP handshakes with the 5G gateway every 30 seconds, sends successful ICMP pings to the internet every 5 seconds with 10ms latency, and successfully completed real application traffic including NTP time synchronization through the 5G network. Zero packet loss was recorded throughout the entire capture."

7. AT Command Logging (Cellular Module Activity)

AT commands are direct instructions sent to the RM520N-GL 5G module through the serial port /dev/ttyUSB3. They allow reading signal strength, network status, and connection information directly from the module hardware.

- **The at_logger.py Script**

Script location: /home/pi/at_logger.py

Log location: /home/pi/network_logs/at_log_YYYYMMDD_HHMMSS.txt

Frequency: Every 60 seconds

```
pi@raspberrypi:~$ ls -t /home/pi/network_logs/at_log_*.txt | head -1
/home/pi/network_logs/at_log_20260615_162109.txt
pi@raspberrypi:~$ cat $(ls -t /home/pi/network_logs/at_log_*.txt | head -1)
=====
AT Command Logger Started
Date: 2026-06-15 16:21:09.971748
=====

--- 2026-06-15 16:21:10.973214 ---
Signal Strength: AT+CSQ
AT+CSQ
+CSQ: 99,99

OK

5G Registration: AT+C5GREG?
AT+C5GREG?
+C5GREG: 0,1

OK

Serving Cell Info: AT+QENG="servingcell"
AT+QENG="servingcell"
+QENG: "servingcell","NOCNN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-72,-10,30,1,-

OK

SIM Status: AT+CPIN?
AT+CPIN?
+CPIN: READY

OK

=====

--- 2026-06-15 16:22:20.377682 ---
Signal Strength: AT+CSQ
AT+CSQ
+CSQ: 99,99

OK

5G Registration: AT+C5GREG?
AT+C5GREG?
+C5GREG: 0,1

OK

Serving Cell Info: AT+QENG="servingcell"
AT+QENG="servingcell"
+QENG: "servingcell","NOCNN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-73,-10,31,1,-

OK

SIM Status: AT+CPIN?
AT+CPIN?
+CPIN: READY

OK

=====
```

=====

--- 2026-06-15 16:23:29.782308 ---

Signal Strength: AT+CSQ

AT+CSQ

+CSQ: 99,99

OK

5G Registration: AT+C5GREG?

AT+C5GREG?

+C5GREG: 0,1

OK

Serving Cell Info: AT+QENG="servingcell"

AT+QENG="servingcell"

+QENG: "servingcell","NOCNN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-70,-10,31,1,-

OK

SIM Status: AT+CPIN?

AT+CPIN?

+CPIN: READY

OK

=====

--- 2026-06-15 16:24:39.187091 ---

Signal Strength: AT+CSQ

AT+CSQ

+CSQ: 99,99

OK

5G Registration: AT+C5GREG?

AT+C5GREG?

+C5GREG: 0,1

OK

Serving Cell Info: AT+QENG="servingcell"

AT+QENG="servingcell"

+QENG: "servingcell","NOCNN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-70,-10,33,1,-

OK

SIM Status: AT+CPIN?

AT+CPIN?

+CPIN: READY

OK

=====

--- 2026-06-15 16:25:48.591486 ---

Signal Strength: AT+CSQ

AT+CSQ

+CSQ: 99,99

OK

5G Registration: AT+C5GREG?

AT+C5GREG?

+C5GREG: 0,1

OK

Serving Cell Info: AT+QENG="servingcell"

AT+QENG="servingcell"

+QENG: "servingcell","NOCNN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-71,-10,32,1,-

OK

SIM Status: AT+CPIN?

AT+CPIN?

+CPIN: READY

OK

=====

```
=====
--- 2026-06-15 16:26:57.995804 ---
Signal Strength: AT+CSQ
AT+CSQ
+CSQ: 99,99
OK

5G Registration: AT+C5GREG?
AT+C5GREG?
+C5GREG: 0,1
OK

Serving Cell Info: AT+QENG="servingcell"
AT+QENG="servingcell"
+QENG: "servingcell","NOCONN","NR5G-SA","TDD",244,38,000FB4001,150,3E8,663264,77,12,-71,-10,32,1,-
OK

SIM Status: AT+CPIN?
AT+CPIN?
+CPIN: READY
OK

=====
pi@raspberrypi:~$
```

• **AT Commands Used and Results**

AT Command	What It Logs	Result Observed
AT+CSQ	Signal strength (legacy)	+CSQ: 99,99 (normal for 5G - CSQ is designed for 2G/3G)
AT+C5GREG?	5G network registration	+C5GREG: 0,1 (Registered on 5G - never changed)
AT+QENG="servingcell"	Full 5G signal information	NR5G-SA, TDD, Band n77, RSRP -70 to -73, SINR 30-33
AT+CPIN?	SIM card status	+CPIN: READY (always ready)

• **Signal Readings Over Time**

Time	RSRP	SINR	5G Registered	SIM Status
16:21:10	-72 dBm	30 dB	yes	READY
16:22:20	-73 dBm	31 dB	yes	READY
16:23:29	-70 dBm	31 dB	yes	READY
16:24:39	-70 dBm	33 dB	yes	READY
16:25:48	-71 dBm	32 dB	yes	READY
16:26:57	-71 dBm	32 dB	yes	READY

Signal Quality: RSRP -70 to -73 dBm = Good. SINR 30-33 dB = Excellent.

8. Connection Loss Monitoring and Testing

- **The connection_monitor.sh Script**

Script location: /home/pi/connection_monitor.sh

Log location: /home/pi/network_logs/connection_loss_YYYYMMDD_HHMMSS.txt

Check frequency: Every 10 seconds

What the script monitors:

- Pings 8.8.8.8 through the 5G interface (usb0) every 10 seconds
- Detects when connection drops - logs exact date and time
- Records interface status at the moment of disconnection
- Detects when connection restores - logs exact date and time
- Counts total number of connection drops

- **Long-term Monitoring Results**

The connection monitor was run on two separate test days: June 10, 2026, and June 15, 2026.

Test Day 1 - June 10, 2026:

Run it with the command, bash /home/pi/network_logger.sh

```
pi@raspberrypi:~ $ bash /home/pi/network_logger.sh
=====
5G Network Logger Started
Date: Wed 10 Jun 13:28:47 EEST 2026
Logs saved to: /home/pi/network_logs
=====
Starting packet capture on usb0...
Starting ping logger...
Starting signal strength logger...
Starting connection status logger...
All loggers running!
Press Ctrl+C to stop
Capturing on 'usb0'
44 |

pi@raspberrypi:~ $ ls /home/pi/network_logs/
5g_traffic_20260610_132847.pcap ping_log_20260610_132847.txt signal_log_20260610_132847.txt status_log_20260610_132847.txt
```

- Traffic captured on June 10
- AT command logs running
- Connection monitoring running
- Ping logs collected

Test Day 2 - June 15, 2026:

```
pi@raspberrypi:~ $ bash /home/pi/network_logger.sh
=====
5G Network Logger Started
Date: Mon 15 Jun 10:11:36 EEST 2026
Logs saved to: /home/pi/network_logs
=====
Starting packet capture on usb0...
Starting ping logger...
Starting signal strength logger...
Starting connection status logger...
All loggers running!
Press Ctrl+C to stop
[sudo] password for pi: Capturing on 'usb0'
18 |
```

```
pi@raspberrypi:~ $ ls /home/pi/network_logs/
5g_traffic_20260615_132847.pcap  at_logger_output.txt  ping_log_20260615_153649.txt  startup.txt
5g_traffic_20260615_150730.pcap  connection_loss_20260615_105136.txt  ping_log_20260615_154143.txt  status_log_20260615_132847.txt
5g_traffic_20260615_153649.pcap  connection_loss_20260615_153649.txt  ping_log_20260615_180755.txt  status_log_20260615_145608.txt
5g_traffic_20260615_154604.pcap  connection_loss_20260615_154143.txt  ping_log_20260615_181136.txt  status_log_20260615_150730.txt
5g_traffic_20260615_092111.pcap  connection_loss_20260615_154604.txt  signal_log_20260615_132847.txt  status_log_20260615_153649.txt
5g_traffic_20260615_101136.pcap  connection_loss_20260615_092111.txt  signal_log_20260615_145608.txt  status_log_20260615_154143.txt
at_log_20260615_141925.txt        monitor_output.txt      signal_log_20260615_150730.txt  status_log_20260615_180755.txt
at_log_20260615_153649.txt        network_output.txt      signal_log_20260615_153649.txt  status_log_20260615_181136.txt
at_log_20260615_154143.txt        ping_log_20260615_132847.txt  signal_log_20260615_154143.txt  tshark_output.txt
at_log_20260615_154604.txt        ping_log_20260615_145608.txt  signal_log_20260615_180755.txt
at_log_20260615_092111.txt        ping_log_20260615_150730.txt  signal_log_20260615_181136.txt
```

Logs auto started after boot:

- 5g_traffic_20260615_092111.pcap - started at 09:21
- 5g_traffic_20260615_101136.pcap - started at 10:11
- at_log_20260615_092111.txt
- signal_log_20260615_100755.txt
- connection_loss_20260615_092111.txt
- ping_log_20260615_100755.txt

The private 5G network was monitored on two separate test days (June 10 and June 15, 2026). Zero unintended connection drops were observed during both test sessions. The network demonstrated perfect stability across both testing periods.

• Connection Loss Test

A deliberate connection loss test was performed to verify the monitoring script works correctly.

Test method:

- Two SSH terminals were opened simultaneously
- First terminal watched the live log using: tail -f connection_loss_20260615_160542.txt
- Second terminal deliberately brought usb0 down: sudo ip link set usb0 down
- After 2 minutes, connection was restored: sudo ip link set usb0 up

Test results:

Event	Time	Detected	Logged
Connection dropped (usb0 down)	16:09:03	Within 10 seconds	Yes
Drop number counted	16:09:03	Drop #1	Yes
Interface status at drop	16:09:03	usb0 state DOWN	Yes
Connection restored (usb0 up)	16:10:53	Within 10 seconds	Yes
Total downtime logged	1 min 50 sec	Accurate	Yes

Sample log output:

Make sure connection monitor is running: `ps aux | grep connection_monitor`

```
pi@raspberrypi:~$ ps aux | grep connection_monitor
pi      14651  0.0  0.0   7088  3280 pts/0    S   14:34   0:00 bash /home/pi/connection_monitor.sh
pi      14933  0.0  0.0   6560  2160 pts/0    S+  14:36   0:00 grep --color=auto connection_monitor
```

Open second terminal and log it in ssh.

`ssh pi@raspberrypi.local`

```
PS C:\Users\shami> ssh pi@raspberrypi.local
pi@raspberrypi.local's password:
Permission denied, please try again.
pi@raspberrypi.local's password:
Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 11:18:37 2026 from fe80::88ae:58e4:418:a6de%wlan0
pi@raspberrypi:~$ |
```

```
pi@raspberrypi:~$ tail -f /home/pi/network_logs/connection_loss_20260615_160542.txt
Mon 15 Jun 16:06:23 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:06:33 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:06:43 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:06:53 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:03 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:13 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:23 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:33 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:43 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:07:53 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:03 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:13 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:23 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:33 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:43 EEST 2026: SG Status - Connected: true - Total drops: 0
Mon 15 Jun 16:08:53 EEST 2026: SG Status - Connected: true - Total drops: 0

CONNECTION LOST: Mon 15 Jun 16:09:03 EEST 2026
Drop number: 1
Interface status:
3: usb0: <BROADCAST,MULTICAST> mtu 1358 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 3a:b7:9f:e2:3a:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.225.97/22 brd 192.168.227.255 scope global dynamic noprefixroute usb0
        valid_lft 41871sec preferred_lft 41871sec
    inet6 fe80::3ab7:9fff:fe2:3a5d/64 scope link tentative noprefixroute
        valid_lft forever preferred_lft forever
Signal at time of drop:
Signal logging skipped
=====
Mon 15 Jun 16:09:03 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:09:13 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:09:23 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:09:33 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:09:43 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:09:53 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:10:03 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:10:13 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:10:23 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:10:33 EEST 2026: SG Status - Connected: false - Total drops: 1
Mon 15 Jun 16:10:43 EEST 2026: SG Status - Connected: false - Total drops: 1

CONNECTION RESTORED: Mon 15 Jun 16:10:53 EEST 2026
Was disconnected since: Mon 15 Jun 16:09:03 EEST 2026
Total drops so far: 1
=====
Mon 15 Jun 16:10:53 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:03 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:13 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:23 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:33 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:43 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:11:53 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:12:03 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:12:13 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:12:23 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:12:33 EEST 2026: SG Status - Connected: true - Total drops: 1
Mon 15 Jun 16:12:43 EEST 2026: SG Status - Connected: true - Total drops: 1
```

```
pi@raspberrypi:~$ sudo ip link set usb0 down
pi@raspberrypi:~$ sudo ip link set usb0 up
pi@raspberrypi:~$
```

9. Auto-Start on Boot Configuration

All 3 logging scripts were configured to start automatically every time the Raspberry Pi boots up, so logging never stops even after power cuts or reboots

- **The start_loggers.sh Script**

Script location: /home/pi/start_loggers.sh

What it does:

- Waits 30 seconds after boot for network to be ready
- Starts at_logger.py (Python AT command logger) in background
- Starts connection_monitor.sh in background
- Starts tshark packet capture on usb0 in background
- Logs startup time to startup.txt

- **Crontab Configuration**

The following line was added to crontab to run on every boot:

```
@reboot bash /home/pi/start_loggers.sh
```

```

pi@raspberrypi:~$ nano /home/pi/start_loggers.sh
pi@raspberrypi:~$ chmod +x /home/pi/start_loggers.sh
pi@raspberrypi:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
@reboot bash /home/pi/start_loggers.sh

```

Then Reboot, wait sometimes for SSH back, and then check whether that all script is running by `ps aux | grep -E "at_logger|connection|tshark|ping"`

```

pi@raspberrypi:~$ sudo reboot
pi@raspberrypi:~$ client_loop: send disconnect: Connection reset

C:\Users\shami>ssh pi@raspberrypi.local
pi@raspberrypi.local's password:
Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 15:25:23 2026 from fe80::88ae:58e4:418:a6de%wlan0
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 15:25:23 2026 from fe80::88ae:58e4:418:a6de%wlan0
pi@raspberrypi:~$ ps aux | grep -E "at_logger|connection|tshark|ping"
pi 1860 0.0 0.0 18288 18616 ? S 16:21 0:00 python3 /home/pi/at_logger.py
pi 1861 0.0 0.0 7888 3264 ? S 16:21 0:00 bash /home/pi/connection_monitor.sh
pi 1863 0.2 0.9 572752 157328 ? Sl 16:21 0:00 tshark -i usb0 -w /home/pi/network_logs/bg_traffic_20260615_162109.pcap
pi 2029 0.0 0.0 6568 2112 pts/0 S+ 16:22 0:00 grep --color=auto -E at_logger|connection|tshark|ping

```

- **Startup Log History**

Boot #	Data	Time	Status
1	Wed 10 Jun 2026	15:36:49	All loggers started
2	Wed 10 Jun 2026	15:46:04	All loggers started
3	Mon 15 Jun 2026	09:21:11	All loggers started
4	Mon 15 Jun 2026	16:21:09	All loggers started

Auto-start has worked correctly on every single boot since June 10, 2026

```

pi@raspberrypi:~$ cat /home/pi/network_logs/startup.txt
All loggers started at Wed 10 Jun 15:36:49 EEST 2026
All loggers started at Wed 10 Jun 15:46:04 EEST 2026
All loggers started at Mon 15 Jun 09:21:11 EEST 2026
All loggers started at Mon 15 Jun 16:21:09 EEST 2026
All loggers running at Mon 15 Jun 16:21:09 EEST 2026
pi@raspberrypi:~$

```

10. Key Notes and Observations

Topic	Note
Raspberry Pi 5 Compatibility	HAT is fully compatible with Pi 5. Metal case was also removed as it only fits Pi 4B. HAT connected to Pi via USB 3.0 dual port adapter.
Active Cooler Conflict	Raspberry Pi 5 Active Cooler was removed as it conflicted with the HAT board.
External Power	5V 3A external power to HAT is mandatory. Pi's USB port alone cannot power the 5G module reliably.
DIP Switch Setting	Switch C must be ON for RM520N-GL (RM50X series). Wrong setting = module not detected.
SIM Card	Nano SIM must be pre-provisioned for the lab's private 5G network by the network administrator.
AT Command Port	ttyUSB2 is used for minicom. ttyUSB3 is used for at_logger.py — they cannot share the same port simultaneously.
Dual Connectivity	5G and WiFi operate simultaneously and independently — 5G works even when WiFi is completely disabled.
Headless Setup	SSH is used for all configurations. No monitor or keyboard needed after initial OS flash.
USB Mode	ECM mode (AT+QCFG=usbnet,1) used for Linux — creates usb0 network interface automatically.

Project Goals Achieved

- Raspberry Pi 5 successfully configured with WaveShare RM520N-GL 5G HAT
- Connected to lab private 5G NR Standalone network (operator FXR, Finland)
- 5G internet connectivity validated with 0% packet loss
- Download speed 182.98 Mbps and upload speed 154.99 Mbps confirmed
- 5G performance exceeds lab WiFi on all metrics
- Network traffic logged and analysed (tshark pcap capture)
- ARP handshakes, ICMP ping, NTP and ICMPv6 traffic identified
- AT command logging running every 60 seconds
- Connection loss monitoring and testing completed successfully
- All scripts auto-start on boot via crontab
- Device ready for 5G network testing and debugging